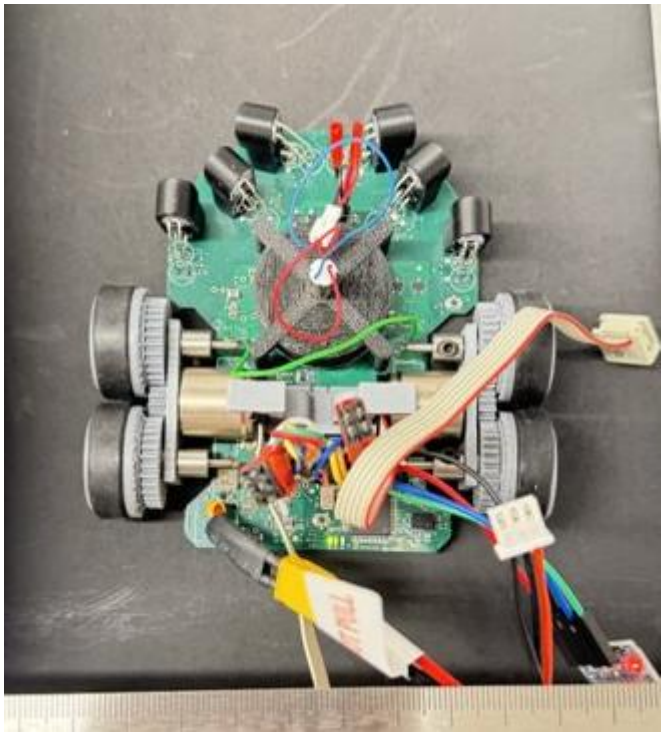
1. For MicroMouse, I jointly developed the full competition firmware for an autonomous maze-solving robot on an STM32. What I'm most proud of is how all the sensors came together into a working navigation system. Each sensor served a different purpose: IR sensors provided wall distance, the IMU provided heading correction, the encoders on the motors gave distance calculations, and the Bluetooth sensor provided live sensor information for debugging. The coolest part of it all, however, was understanding the robot as a system and figuring out exactly how each of those peripherals interacted with one another within the flood-fill navigation algorithm, ultimately forming a maze-solving machine. Getting that mental model not only made everything click for me, but also made me think of embedded systems completely differently.



2. My favorite feature of microcontrollers is honestly timers. Timers are an underrated pick/feature that is usually abstracted away for most MCUs, but they form the core of nearly every major feature of any microcontroller. They can simultaneously generate PWM signals, read encoder pulses, and trigger ADC conversions, without any CPU involvement. The fact that a single piece of hardware can be the core of so many features is something I find really cool about microcontrollers.
3. We wanted to implement a gesture-based machine learning algorithm, but for the microcontrollers that we ended up using, we didn't know that the battery we used up the I2C pins to send battery state information into the MCU. Then, when we needed to use the I2C pins for the IMU device, we realized we had no solution. Instead of trying to redo the hardware or give up altogether, I bit-banged I2C/implemented software I2C on some of the non-I2C-compatible pins to get information from the IMU. This wasn't a great fix, since now our IMU had to run significantly slower, since the pins were only able to handle low speeds, but, it was enough to keep the project going.
4. The first thing I'd want to know would be the application of the PCB being used, for example, high-speed circuits run with a significant range of issues differing from low-

speed circuits; things like parasitic capacitances and impedance mismatching become major issues now. That said, my first two hardware checks would still be the same. I would probe the power rails of all ICs and MCUs and ensure that during run-time, they are within the operating range, including voltage fluctuations, closely referencing the corresponding data sheets. My first instinct when I see an issue like this is that voltage fluctuations may cause the voltage to dip below the required threshold for a component to run, causing it to reset. Then I would specifically check the RESET pins of any ICs and MCUs, ensuring that they aren't being pulled low unexpectedly due to circuit issues.

5. My strongest category is embedded firmware; I've built production-level embedded C on an STM32 managing multiple peripherals simultaneously, including I2C, UART, ADCs, and PWM, and I'm comfortable working directly with data sheets to configure hardware from scratch. I also have close experience with PCB design and hardware, I've owned projects end-to-end, from schematic to final assembly for the design of a servo controller board which incorporated a buck converter and the bare ATmega chip which is what separates me from having a pure firmware background. If something is behaving irregularly, I'm comfortable probing power rails and signal lines just as much as stepping through firmware code in a debugger.